

Sección 1 — Lectura

Plan de aprendizaje práctico

Cómo dominar (y saber construir tú mismo) la sección de lectura y su pipeline de PLN

Ruta progresiva de 8 fases, de lo básico a un proyecto final. Cada fase trae objetivo, conceptos, una **mini-implementación** para aprender haciendo, un ejemplo de código, recursos y un criterio de dominio. Al final, un **capstone** que recrea de punta a punta una versión mínima de la sección de lectura.

Cómo usar este plan

- **Aprende haciendo.** No pases de fase sin completar su mini-implementación; el objetivo no es leer, es **saber construirlo**.
- **Orden sugerido:** las fases 1-4 (frontend) y 5-8 (Python/PLN) son bastante independientes; puedes alternarlas, pero dentro de cada bloque respeta el orden.
- **Dos velocidades:** ruta rápida = solo mini-implementaciones + capstone (~3-4 semanas a tiempo parcial); ruta profunda = añade los recursos largos y los retos extra.
- **Ten el repo a la vista.** Cada fase corresponde a archivos reales del proyecto *indescifrable*; ábrelos como “solución de referencia” tras intentar tu versión.

Mapa de fases

Fase	Tema	Archivos de referencia en el repo
0	Prerrequisitos: JS moderno, Node, Git	—
1	React: componentes, estado, efectos	Landing.jsx, Tablero.jsx
2	Persistencia + funciones puras + pruebas	engine/bolsa.js, *.test.js
3	Enrutado y estructura de la app	App.jsx, Biblioteca.jsx
4	Datos y experiencia de lectura	Lector.jsx, data/lecturas/
5	Python + spaCy (segmentar, lematizar)	pipeline/procesar.py
6	Diccionarios y traducción por palabra	pipeline/traductor.py, leer_diccionario.py
7	Traducción por oración: MT y LLM	pipeline/mt.py, importar_traducccion.py
8	Integración del pipeline y despliegue	construir_lexico.py, Netlify/git
Cap.	Capstone: mini-lector de punta a punta	(todo junto)

Fase 0 — Prerrequisitos: JavaScript moderno, Node y Git

Objetivo: manejar con soltura el JavaScript que usa React y las herramientas de base. Sin esto, todo lo demás cuesta el doble.

Conceptos clave

- ES6+: `const/let`, funciones flecha, **destructuring**, `spread ...`, template strings, módulos `import/export`.
- Métodos de arreglo: `map`, `filter`, `reduce`, `find`, `some` (imprescindibles para listas en React).
- Asincronía: `Promise`, `async/await`, `fetch`.
- Node + npm (instalar dependencias, scripts) y Git (`commit`, `push`, ramas).

MINI-IMPLEMENTACIÓN — Calentamiento sin framework

Crea un archivo `bolsa.js` con funciones puras que operen sobre un arreglo de palabras y pruébalas con `console.log`:

- `agregar(bolsa, palabra)` que no duplique.
- `quitar(bolsa, palabra)` y `contar(bolsa)`.
- Reto: que `agregar` devuelva un **nuevo** arreglo (inmutabilidad), sin mutar el original.

```
export const agregar = (bolsa, p) =>
  bolsa.includes(p) ? bolsa : [...bolsa, p];

console.log(agregar(["sol"], "luna")); // ["sol", "luna"]
console.log(agregar(["sol"], "sol")); // ["sol"] (no duplica)
```

Recursos

- **javascript.info** — <https://javascript.info>
El mejor tutorial moderno de JS, gratis.
- **MDN JavaScript** — <https://developer.mozilla.org/es/docs/Web/JavaScript>
Referencia. Busca map/filter/reduce, destructuring, async/await.
- **Node.js** — <https://nodejs.org>
Instala la versión LTS; trae npm.
- **Git handbook** — <https://docs.github.com/get-started>
commit, push, ramas.

SABRÁS QUE LO DOMINAS CUANDO

Puedes escribir funciones que reciben datos y devuelven datos nuevos sin mutar, y explicar la diferencia entre map y forEach.

Fase 1 — React: componentes, estado y efectos

Objetivo: construir interfaces por componentes y entender **cuándo** y **por qué** se vuelven a renderizar.

Conceptos clave

- Componente = función que devuelve JSX. Props (datos que entran) vs estado (useState).
- Renderizado condicional ({cond && ...}) y listas con .map() + key estable.
- useEffect para efectos (cargar datos, temporizadores) y su array de dependencias.
- Eventos: onClick, onChange; subir el estado al componente padre.

MINI-IMPLEMENTACIÓN — Bolsa de palabras en pantalla

Monta un proyecto con Vite (npm create vite@latest -- --template react) y crea una **Bolsa**: un input + botón que añade palabras a una lista en useState, con botón para quitar cada una.

- Usa .map() con key para la lista.
- Reto: importa tu agregar/quitar de la Fase 0 y úsalos (lógica separada de la UI).

```
function Bolsa() {
  const [palabras, setPalabras] = useState([]);
  const [texto, setTexto] = useState("");
  return (
    <div>
      <input value={texto} onChange={e => setTexto(e.target.value)} />
      <button onClick={() => setPalabras(agregar(palabras, texto))}>Añadir</button>
      <ul>{palabras.map(p => <li key={p}>{p}</li>)}</ul>
    </div>
  );
}
```

Recursos

- **react.dev — Learn** — <https://react.dev/learn>
Tutorial oficial. Haz "Tic-Tac-Toe" y "Thinking in React".

- **react.dev — useEffect** — <https://react.dev/learn/synchronizing-with-effects>
Cuándo (y cuándo NO) usar efectos.
- **Vite — Guide** — <https://vite.dev/guide/>
Crear el proyecto, dev server, build.

SABRÁS QUE LO DOMINAS CUANDO

Puedes explicar por qué React necesita key en las listas y predecir cuándo se re-renderiza un componente.

Fase 2 — Persistencia, funciones puras y pruebas

Objetivo: separar la **lógica pura** (testable) de los **efectos** (localStorage, DOM). Es el patrón del “motor” del proyecto y el germen del núcleo matemático de la tesis.

IDEA CLAVE — Puro vs impuro

Una función **pura** no toca el mundo exterior: misma entrada -> misma salida, sin efectos. `agregarPalabra(bolsa, x)` es pura; `localStorage.setItem(...)` es un efecto. Separarlos permite **probar** la lógica en aislamiento y reutilizarla.

Conceptos clave

- **localStorage:** guardar/leer JSON (adaptador delgado e impuro).
- **Módulo puro:** la regla del proyecto es “una palabra ya presente **conserva su estado**”.
- **Pruebas con Vitest:** `describe/it/expect`.

MINI-IMPLEMENTACIÓN — Motor de la bolsa + su prueba

Extrae `agregarPalabra/quitarPalabra` a `bolsa.js`, un `almacenamiento.js` que persista en `localStorage`, y escribe una prueba Vitest.

- **Test:** agregar una palabra nueva la añade; agregar una existente **no** la duplica ni cambia su estado.
- **Instala y corre:** `npm i -D vitest y npx vitest`.

```
import { describe, it, expect } from "vitest";
import { agregarPalabra } from "../bolsa";

it("no duplica ni reinicia una palabra existente", () => {
  const previa = { id: "de:frau", caja: 4 }; // estado acumulado
  const bolsa = agregarPalabra([previa], { id: "de:frau", caja: 0 });
  expect(bolsa).toHaveLength(1);
  expect(bolsa[0].caja).toBe(4); // conserva el estado
});
```

Recursos

- **Vitest** — <https://vitest.dev/guide/>
Configuración y API de pruebas.
- **MDN localStorage** — <https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>
`getItem/setItem` + `JSON.parse/stringify`.
- **react.dev — Keeping components pure** — <https://react.dev/learn/keeping-components-pure>
Por qué la pureza importa.

SABRÁS QUE LO DOMINAS CUANDO

Puedes escribir una función pura, su adaptador de persistencia y una prueba que verifique una regla de negocio.

Fase 3 — Enrutado y estructura de la aplicación

Objetivo: convertir componentes sueltos en una app con secciones y URLs.

Conceptos clave

- `react-router-dom`: `BrowserRouter`, `Routes`, `Route`, `Link`, `useParams`, `useNavigate`.
- Parámetros de ruta (`/lectura/:id`) y de búsqueda (`useSearchParams`) para conservar filtros.
- Fallback SPA en el hosting (`_redirects` en Netlify) para que las URLs profundas no den 404.

MINI-IMPLEMENTACIÓN — Biblioteca -> Detalle

Crea 3 rutas: inicio, una lista de elementos, y un detalle `/item/:id`. Al hacer clic en la lista, navega al detalle y lee el `:id` con `useParams`.

- Guarda un filtro (idioma/nivel) en la URL con `useSearchParams` y comprueba que se conserva al volver.

```
<Routes>
  <Route path="/" element={<Home/>} />
  <Route path="/lectura" element={<Biblioteca/>} />
  <Route path="/lectura/:idioma/:nivel/:id" element={<Lector/>} />
</Routes>
// en Lector: const { idioma, nivel, id } = useParams();
```

Recursos

- **React Router — Tutorial** — <https://reactrouter.com/start/tutorial>
Rutas, params, navegación.
- **Netlify — Redirects** — <https://docs.netlify.com/routing/redirects/>
El fallback `/* /index.html 200` para SPA.

SABRÁS QUE LO DOMINAS CUANDO

Puedes montar una app multi-página con URLs compartibles y leer parámetros de la ruta.

Fase 4 — Modelado de datos y la experiencia de lectura

Objetivo: el corazón del frontend: renderizar un texto con palabras clicables, mostrar traducciones y la traducción por frase.

Conceptos clave

- Diseñar el JSON de una lectura: cuerpo como arreglo de frases **alineadas** por índice entre idiomas.
- **Tokenización** con regex Unicode (`\p{L}`) que separa palabras de puntuación conservando el texto.
- Mapear un clic de palabra a una entrada de un **léxico** (`idioma:forma -> traducción`).
- Cargar datos con `import.meta.glob` y el léxico grande con `dynamic import` (peso del bundle).

MINI-IMPLEMENTACIÓN — Mini-lector

Con un objeto JS `{ de:[...], es:[...] }` y un mini-léxico `{ 'de:frau':'mujer', ... }`, renderiza el texto alemán con cada palabra clicable; al hacer clic, muestra su traducción; y un botón por frase que revela la línea española alineada.

- Escribe `tokenizar(frase)` con `String.matchAll` y una regex Unicode.
- Reto: resalta las palabras que ya guardaste en la bolsa.

```
function tokenizar(frase) {
  const re = /\p{L}[\p{L}\p{M}']-*/gu;    // 'u' = unicode
  const out = []; let last = 0, m;
  while ((m = re.exec(frase)) !== null) {
    if (m.index > last) out.push({tipo:"sep", v: frase.slice(last, m.index)});
    out.push({tipo:"palabra", v: m[0]});
    last = m.index + m[0].length;
  }
  if (last < frase.length) out.push({tipo:"sep", v: frase.slice(last)});
  return out;
}
```

Recursos

- **MDN — Unicode property escapes** — https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Regular_expressions/Unicode_character_class_escapes
La clave de \p{L} y la bandera u.
- **Vite — Glob import** — <https://vite.dev/guide/features.html#glob-import>
import.meta.glob (eager y lazy) y dynamic import.

SABRÁS QUE LO DOMINAS CUANDO

Puedes convertir una frase en palabras clicables y conectarlas a un diccionario en memoria, con la traducción por frase alineada.

Fase 5 — Python + spaCy: segmentar, tokenizar, lematizar

Objetivo: el primer paso del pipeline. Convertir texto crudo en frases y en tokens con su lema y categoría gramatical.

Conceptos clave

- Entorno de Python: venv + pip; instalar spaCy y un modelo (`python -m spacy download de_core_news_md`).
- El objeto `doc = nlp(texto)`: iterar tokens; `token.lemma_`, `.pos_`, `.dep_`; frases con `doc.sents`.
- Ambigüedad morfológica y su tratamiento algorítmico (los **verbos separables** del alemán vía la dependencia svp).

MINI-IMPLEMENTACIÓN — Lemas y verbos separables

Escribe un script que imprima, para cada token de una frase alemana, su forma y su lema. Luego detecta la dependencia svp y **reconstruye** el verbo separable.

- Frase de prueba: “...bereitet sie das Frühstück ... vor.” -> el lema real de *bereitet* es **vorbereiten**.
- Reto: cuenta cuántas frases produce spaCy en un párrafo largo y compara con las que esperabas (verás el problema de las líneas envueltas).

```
import spacy
nlp = spacy.load("de_core_news_md")
doc = nlp("Zu Hause bereitet sie das Frühstück für ihre Familie vor.")
lemas = {t.i: t.lemma_ for t in doc}
for t in doc:
    if t.dep_ == "svp":
        # prefijo separable
        lemas[t.head.i] = t.text.lower() + t.head.lemma_ # vor+bereiten
print(lemas) # ... 'vorbereiten' ...
```

Recursos

- **spaCy — Curso gratis** — <https://course.spacy.io/es>
Curso interactivo oficial (¡en español!). El mejor punto de entrada.
- **spaCy 101** — <https://spacy.io/usage/spacy-101>
Conceptos y objetos base.
- **spaCy — Linguistic features** — <https://spacy.io/usage/linguistic-features>
POS, lemas, dependencias, morfología.

SABRÁS QUE LO DOMINAS CUANDO

Puedes cargar un modelo, extraer frases y lemas, e inspeccionar dependencias para resolver un caso morfológico.

Fase 6 — Diccionarios y traducción por palabra

Objetivo: traducir palabra por palabra **sin API**, y aprender a medir y mejorar la cobertura.

Conceptos clave

- Parsear un diccionario **FreeDict** (TEI/XML) con `xml.etree.ElementTree`; cuidado con los **namespaces** XML.
- Estrategia por capas: directo -> **cadena** por un idioma puente (de->en->es) -> **descomposición de compuestos**.
- **Cobertura**: % de formas con traducción; diagnóstico por categoría gramatical para saber qué falla y por qué.

IDEA CLAVE — Medir antes de optimizar

En el proyecto, la cobertura pasó de 70% a 92%. El diagnóstico reveló que el problema no era la lematización sino el **tamaño del diccionario**: por eso la solución fue la cadena por inglés (el diccionario de->en es enorme), no perseguir mejoras marginales del lematizador.

MINI-IMPLEMENTACIÓN — Parser + traductor con fallback

Descarga un FreeDict pequeño (p. ej. deu-spa), parsea las entradas a un dict lema->traducción, y escribe `traducir(lema, forma)` que pruebe: lema, luego la forma, y sobre una lista de 50 palabras **mide la cobertura**.

- Reto: añade la cabeza del compuesto (sufijo ≥ 3 letras) como último fallback.

```
import xml.etree.ElementTree as ET
NS = "{http://www.tei-c.org/ns/1.0}"
raiz = ET.parse("deu-spa.tei").getroot()
dic = {}
for e in raiz.iter(f"{NS}entry"):
    orth = e.find(f".//{NS}form/{NS}orth")
    trads = [q.text for c in e.iter(f"{NS}cit") if c.get("type")=="trans"
              for q in c.findall(f"{NS}quote")]
    if orth is not None and trads:
        dic.setdefault(orth.text.lower(), " / ".join(trads[:3]))
print(dic.get("frau")) # mujer / esposa / señora
```

Recursos

- **FreeDict** — <https://freedict.org>
Diccionarios libres; descarga el formato 'src' (TEI).

- **Python — ElementTree** — <https://docs.python.org/3/library/xml.etree.elementtree.html>
Parseo de XML; ojo con los namespaces {...}.
- **Wiktionary / kaikki** — <https://kaikki.org>
Alternativa de mayor cobertura (más pesada).

SABRÁS QUE LO DOMINAS CUANDO

Puedes parsear un diccionario, construir un traductor con fallbacks y reportar su cobertura sobre un texto.

Fase 7 — Traducción por oración: MT y LLM

Objetivo: dar traducción por frase a un libro conservando la **alineación 1:1**, por dos vías, y entender sus compromisos.

Conceptos clave

- El requisito duro: la frase es[i] debe corresponder a de[i].
- **MT offline** con transformers + opus-mt (Helsinki-NLP): automática; alineación garantizada (una salida por frase). Evita el **pivote** por inglés si hay modelo directo.
- **LLM** (p. ej. Gemini): mejor calidad pero semi-manual; hay que **validar** la salida (prompt numerado + verificador de conteo).

MINI-IMPLEMENTACIÓN — Traduce 5 frases y valida

Con opus-mt traduce 5 frases alemanas; imprime pares de-es. Luego escribe un **validador** que reciba una traducción numerada de un LLM y compruebe que hay exactamente una línea por frase (si no cuadra, rechaza).

- Compara la misma frase con MT y con un LLM: anota los errores de la MT.

```
from transformers import MarianMTModel, MarianTokenizer
name = "Helsinki-NLP/opus-mt-de-es"
tok, model = MarianTokenizer.from_pretrained(name), MarianMTModel.from_pretrained(name)
def traducir(frases):
    b = tok(frases, return_tensors="pt", padding=True, truncation=True)
    return [tok.decode(g, skip_special_tokens=True) for g in model.generate(**b)]
print(traducir(["Es war kein Traum."])) # ['No fue un sueño.']
```

Recursos

- **HF Transformers** — <https://huggingface.co/docs/transformers/tasks/translation>
Uso de modelos MarianMT/opus-mt.
- **Helsinki-NLP opus-mt** — <https://huggingface.co/Helsinki-NLP>
Modelos de MT por par de idiomas.
- **Argos Translate** — <https://github.com/argosopentech/argos-translate>
MT offline ligera (pivota por inglés; útil conocer sus límites).

SABRÁS QUE LO DOMINAS CUANDO

Puedes traducir un lote de frases con un modelo local y explicar por qué preferirías un LLM o una MT según el caso.

Fase 8 — Integración del pipeline y despliegue

Objetivo: unir todo: de un texto de Gutenberg a JSON que tu app consume, y publicarlo.

Conceptos clave

- Ingesta: descargar de Project Gutenberg, quitar el **boilerplate** y **unir las líneas envueltas** antes de segmentar; trocear en partes legibles.
- Exportar JSON “**pipeline-ready**” con el mismo formato que consume el frontend (no rehacer el frontend después).
- Despliegue: Git + Netlify (build automático); y los baches de entorno (certificados SSL, codificación UTF-8 en consola).

MINI-IMPLEMENTACIÓN — De Gutenberg a tu app

Baja un texto corto de Gutenberg, límpialo, segmentalo con spaCy, exporta un JSON { id, cuerpo:{ de: [. . .] } } y **conéctalo** a tu mini-lector de la Fase 4.

- Añade la traducción por palabra (Fase 6) y por frase con MT (Fase 7).
- Sube a un repo y despliega en Netlify con el fallback SPA.

```
# limpiar líneas envueltas de Gutenberg antes de segmentar
import re, requests, spacy
txt = requests.get("https://www.gutenberg.org/cache/epub/22367/pg22367.txt").text
txt = txt.split("**** END")[0]
txt = txt[txt.find("Als Gregor Samsa"):]
flujo = re.sub(r"\s+", " ", txt).join(txt.splitlines())
frases = [s.text.strip() for s in spacy.load("de_core_news_md")(flujo).sents]
```

Recursos

- **Project Gutenberg** — <https://www.gutenberg.org>
Textos de dominio público (usa pgID.txt).
- **Netlify — Deploy** — <https://docs.netlify.com/site-deploys/create-deploys/>
Conecta el repo; build de Vite automático.
- **pip-system-certs** — <https://pypi.org/project/pip-system-certs/>
Si tienes errores SSL en Windows al descargar modelos/diccionarios.

SABRÁS QUE LO DOMINAS CUANDO

Puedes llevar un texto crudo a JSON consumible por tu app y publicarlo en la web.

Proyecto final (capstone) — Mini-lector de punta a punta

Reúne todo en una versión mínima pero completa de la sección de lectura. No hace falta que sea grande; sí que atraviese **todas** las piezas.

Requisitos

- **Pipeline (Python):** toma 1 texto corto alemán (10-20 frases), lo segmenta con spaCy, genera un léxico de: forma->es con FreeDict (+ fallback) y una traducción por frase con opus-mt. Exporta 1 JSON.
- **Frontend (React):** biblioteca con esa lectura; lector con palabras clicables (traducción por palabra), marcador de traducción por frase, y una **bolsa** persistente en localStorage.
- **Motor puro + prueba:** la lógica de la bolsa en un módulo puro con al menos un test Vitest.
- **Despliegue:** repo en Git + Netlify.

Criterios de éxito

- Puedo **regenerar** el JSON ejecutando el pipeline (reproducible).
- Al tocar una palabra veo su traducción; al pulsar el marcador veo la frase en español.

- La bolsa sobrevive a recargar la página.
- Sé explicar cada decisión (por qué texto paralelo para la frase y léxico para la palabra, por qué separar lo puro de lo impuro, por qué MT vs LLM).

IDEA CLAVE — El orden mental del proyecto

Datos primero, presentación después. Diseña el JSON (el “contrato”) antes de la UI; el pipeline llena ese contrato y el frontend solo lo pinta. Ese desacople es lo que permite cambiar el motor de traducción (FreeDict, MT, LLM) sin tocar la interfaz.

Recursos troncales (para tener a mano)

Tema	Recurso principal
JavaScript	javascript.info
React	react.dev/learn
Enrutado	reactrouter.com
Pruebas	vitest.dev
Bundler	vite.dev/guide
spaCy / PLN	course.spacy.io/es (curso gratis)
Diccionarios	freedict.org
MT	huggingface.co/docs/transformers
Textos	gutenberg.org
Despliegue	docs.netlify.com

Acompaña este plan con el documento “Documentación técnica”, que describe cómo está resuelto cada punto en el proyecto *indescifrable*.